

High-Level Synthesis for Edge-AI Applications

Nader Yahia

Electronic Engineering

Hochschule Hamm-Lippstadt

Lippstadt, Germany

nader.yahia@stud.hshl.de

Abstract—The increasing demand for real-time artificial intelligence inference at the edge introduces strict constraints on latency, power consumption, memory capacity, and communication bandwidth. While FPGAs provide a highly efficient and reconfigurable platform for AI acceleration, traditional RTL-based design introduces significant development complexity and limits rapid hardware exploration. This work investigates High-Level Synthesis (HLS) as a design methodology for FPGA-based Edge AI acceleration by bridging high-level algorithm development and hardware implementation. A 2D convolutional neural network kernel is used as a representative Edge AI workload to demonstrate the impact of HLS optimization directives on the generated hardware architecture. Starting from a sequential baseline implementation, loop pipelining and loop unrolling are applied to explore performance and resource trade-offs. The results demonstrate that HLS enables systematic hardware design space exploration by modifying architectural parameters without changing the functional algorithm. The evaluated designs achieve significant latency reduction while maintaining efficient FPGA resource utilization, highlighting HLS as a practical approach for developing high-performance and resource-aware Edge AI accelerators.

I. INTRODUCTION

Edge Artificial Intelligence (Edge AI) is defined as the execution of machine learning inference directly on endpoint or near-endpoint. It is constrained by on-device training and lightweight adaptation, rather than dependence on cloud round-trip communication [1]. Unlike data-center environments, in which compute resources could be scaled dynamically, edge deployments must strictly satisfy non-negotiable hardware constraints in latency, power, memory, and communication bandwidth. Due to the identified limitations, conventional general-purpose processors, CPUs and GPUs are structurally unsuitable for a broad class of edge inference tasks. Field-Programmable Gate Arrays (FPGAs) offer a reconfigurable hardware fabric that could be tailored precisely to a specific AI workload’s computational structure. Exploring FPGA potential requires mastering HDL, which introduces a significant learning and development overhead. High-Level Synthesis (HLS) removes these barrier to FPGA adoption. It can automatically transform behavioral C/C++ descriptions into synthesizable Register-Transfer Level (RTL) logic [2]. In this section, the system-level motivation for this combined approach will be discussed.

A. Motivation

The feasibility of edge inference platforms is dictated by four interdependent constraints. First, latency requirements

are inherently application-dependent. Use cases such as autonomous collision-avoidance, industrial anomaly detection, and real-time signal classification commonly impose per-inference deadlines in the 1–10 ms range. In such a context, non-deterministic latency introduced by operating-system scheduling, shared cache contention, or DRAM refresh cycles is categorically and strictly unacceptable. Second, power consumption boundaries for edge devices span sub-watt IoT sensor nodes to low tens-of-watts embedded gateways. The efficiency in this regard not defined by touching the lower boudries of consumption, but how efficiently it performs useful AI work under power limits. Third, on-chip memory capacity is limited, leading to frequent data transfers between chip and off the chip. It affects energy and latency penalties that accumulate across layer-wise activation and weight transfers, which in turn makes off-chip memory bandwidth the dominant limiting factor for bandwidth-bound convolutional workloads. [3]. Fourth, off-chip communication bandwidth between the chip and external memory or sensor peripherals dictates how much data can be processed over time, requiring accelerators to minimize external data movement through on-chip buffering and combining multiple consecutive operations into a single execution step(operator fusion).

B. Inadequacy of CPU and GPU Platforms

CPUs are designed to handle a variety of workloads. To achieve this flexibility, they include complex control features like instruction reordering, branch prediction, and large cache systems. That introduces overheads which compromise performance, power consumption, and chip silicon area. For convolutional neural networks, the workload is very repetitive and structured, mainly consisting of multiply–accumulate operations repeated across data. It does not benefit much from the CPU’s advanced control mechanisms, which results in more overhead rather than actual useful computation. For instance, efficient Arm cores operating within a few watts struggle to execute neural network inference fast enough. It typically needs tens to hundreds of milliseconds to process a single frame, which is too slow for real-time applications.

GPUs address MAC-intensive workloads through thousands of SIMD arithmetic units and high-bandwidth memory subsystems. In data-center environments, this delivers exceptional batch throughput, but GPU architectures carry fundamental edge incompatibilities: high-performance GPUs operate at 250

W to 700 W, making them unsuitable for 1 to 10 watt embedded edge applications; GPGPU kernel launch and scheduling overheads introduce latency non-determinism, which is incompatible with the strict timing constraints of embedded edge applications. GPU datapaths lack a reconfigurability feature; in other words, they are fixed at fabrication. That prevents microarchitectural adaptation to a specific model's computational graph. Both processor classes therefore fail to simultaneously satisfy the critical strict requirements of embedded edge applications (e.g., latency, power, and determinism).

II. EDGE AI ACCELERATION STACK

A. FPGA-Based Acceleration

FPGAs consist of configurable logic blocks (CLBs), block RAM (BRAM), DSP slices, high-speed I/O, and a programmable routing network configured through a bitstream. This architecture aligns well with Edge AI requirements due to three key characteristics: First, spatial pipelining enables dedicated hardware execution of inference operations, reducing instruction overhead and improving throughput. Second, custom-precision datapaths allow arithmetic units to match the bit-width requirements of quantized models (e.g., INT4 and INT8), improving computational efficiency. Third, storing weights and activations in on-chip BRAM reduces off-chip memory transfers and mitigates bandwidth limitations [4]. These characteristics make FPGAs suitable for efficient, deterministic, and reconfigurable Edge AI acceleration.

B. High-Level Synthesis as Design Methodology

Mapping a multi-layer CNN to RTL from first principles demands months of engineering effort and deep expertise in digital design, timing closure, and FPGA resource optimization expertise rarely co-located with neural architecture research. This expertise barrier constitutes the principal bottleneck in the Edge AI hardware design cycle.

HLS tools—exemplified by AMD Vitis HLS and Intel HLS Compiler—resolve this barrier by accepting behavioral C/C++ as input and automatically performing scheduling (assignment of operations to clock cycles), binding (allocation of operations to hardware resources), and synthesized RTL generation [2]. Designer control is exercised through directives specifying loop unrolling factors, initiation interval targets, array partitioning, and dataflow decomposition. The consequence is a design-iteration cycle measured in hours rather than weeks: a change to quantization precision or layer parallelism in the C/C++ source yields updated RTL and implementation reports without manual HDL revision. Systematic evaluation demonstrates that HLS-generated RTL achieves 80–95% of the quality of results (QoR) attainable by expert-written RTL for regular MAC-intensive kernels, making it a viable path to production-quality accelerator development [5].

C. System Pipeline Architecture

The end-to-end pipeline from trained AI model to deployed FPGA accelerator comprises five sequential transformation stages, illustrated in Fig. 1.

- **Stage 1: AI Model Specification:** Trained network topology exported from a training framework (e.g., PyTorch / TensorFlow) as an ONNX graph or equivalent intermediate representation. Operator set, weight precision, and quantization scheme fixed.
- **Stage 2: C/C++ Algorithmic Description:** Network operators reimplemented as fixed-point C/C++ functions. Quantized data types applied; operator fusion and layer reordering performed to minimize memory traffic.
- **Stage 3: High-Level Synthesis:** C/C++ source compiled by HLS tool. Scheduling, binding, and datapath generation executed under designer directives. RTL (VHDL/Verilog) and synthesis reports produced automatically.
- **Stage 4: RTL Implementation:** Synthesized RTL processed through technology mapping, place-and-route, and timing closure in FPGA tools (Vivado / Quartus Prime). FPGA bitstream generated.
- **Stage 5: FPGA Execution at the Edge:** Bitstream programmed into target device. Accelerator receives sensor or host data, executes inference deterministically, and returns results within the latency budget.

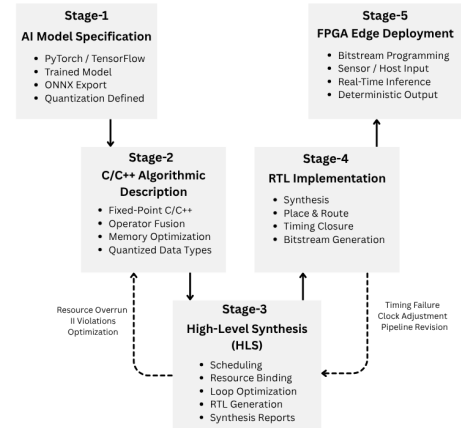


Fig. 1. HLS-based system pipeline.

The pipeline is iterative rather than strictly linear: HLS synthesis reports revealing initiation-interval violations or resource overruns motivate reformulation at Stage 2; timing-closure failures at Stage 4 may necessitate clock frequency adjustment or pipeline depth revision at Stage 3. These feedback loops execute in minutes within an HLS-centric flow versus weeks in an equivalent RTL-centric flow, constituting the primary productivity advantage of the HLS methodology for Edge AI accelerator development. The mathematical and algorithmic foundations of the HLS transformation performed at Stage 3 are formalized in subsequent parts of this study.

III. CNN INFERENCE COMPUTATIONAL MODEL

Efficient hardware acceleration of neural network inference requires a quantitative characterization of the computational workload prior to architectural mapping. Convolutional Neural Networks (CNNs) dominate Edge AI workloads, and their complexity is primarily determined by convolution operations, which are heavily based on multiply-accumulate (MAC) computations [4]. This section provides a compact formulation of the computational model relevant for hardware synthesis.

A. Convolution as Computational Primitive

A convolution layer maps an input feature map $X \in \mathbb{R}^{W \times H \times C_{in}}$ to an output feature map $Y \in \mathbb{R}^{W_{out} \times H_{out} \times C_{out}}$ using kernels of size $K \times K$. The output at position (i, j, k) is defined as:

$$Y(i, j, k) = \sum_{m, n, c} X \cdot W + b \quad (1)$$

where the summation spans the kernel window and input channels. In hardware, this reduces to repeated multiply-accumulate (MAC) operations.

B. MAC Complexity Model

The total number of MAC operations required for a convolution layer is:

$$MAC_{conv} = W_{out} \cdot H_{out} \cdot C_{out} \cdot K^2 \cdot C_{in} \quad (2)$$

This expression directly maps to hardware resource demand in FPGA-based accelerators, where each MAC corresponds to DSP utilization and determines achievable parallelism and throughput. For fully connected layers, the MAC count simplifies to:

$$MAC_{fc} = N_{in} \cdot N_{out} \quad (3)$$

C. Memory and Arithmetic Intensity

Beyond computation, CNN performance is often limited by memory bandwidth. Arithmetic intensity describes the relationship between computation and memory movement. Low arithmetic intensity indicates memory-bound execution, motivating techniques such as on-chip buffering and tiling in FPGA-based HLS designs [6]. In HLS, these characteristics guide parallelization, pipeline scheduling, and buffer organization to optimize MAC throughput under memory constraints.

IV. FORMAL HLS ALGORITHM MODEL

High-Level Synthesis (HLS) constitutes a formal compilation pipeline that transforms a behavioral program specification, expressed in a synthesizable subset of C or C++, into a structurally equivalent RTL hardware description. The transformation is not a simple transcription: the HLS compiler must derive a clocked microarchitecture—including datapath functional units, register assignments, multiplexer networks, and a finite-state machine (FSM) controller—from a description that carries no explicit notion of clock cycles,

hardware resources, or physical timing [7]. For Edge AI applications, this compilation process enables computationally intensive CNN kernels to be translated into dedicated FPGA accelerators without manual RTL development. The pipeline proceeds through five principal stages: behavioral analysis and intermediate representation extraction, scheduling, resource allocation and binding, architectural optimization through HLS directives, and RTL generation.

A. Behavioral Analysis and Dataflow Extraction

Upon parsing the C/C++ source, the HLS front-end constructs intermediate representations that capture both the control structure and computational dependencies of the program. A Control Flow Graph (CFG) models the execution flow, where each node represents a basic block and edges represent possible transfers of control arising from conditional branches, loops, and function calls. Within each basic block, the compiler constructs a Data Flow Graph (DFG), whose nodes represent arithmetic and logical operations while directed edges represent data dependencies between operations. For CNN inference kernels, the DFG explicitly exposes the large number of independent multiply-accumulate (MAC) operations present within convolution loops. This graph serves as the primary representation for hardware synthesis, allowing the compiler to identify parallel operations, determine dependency constraints, and estimate the critical path before hardware generation.

B. Operation Scheduling

Scheduling assigns every operation in the DFG to a specific clock cycle while preserving all data dependencies. Formally, scheduling determines a mapping from operations to control steps under latency and resource constraints [2]. Two classical scheduling algorithms define the feasible scheduling window. As Soon As Possible (ASAP) scheduling computes the minimum execution latency assuming unlimited hardware resources, whereas As Late As Possible (ALAP) scheduling determines the latest execution time of each operation without increasing overall latency. The difference between the ASAP and ALAP schedules defines the operation mobility, while operations with zero mobility form the critical path. In practical FPGA implementations, hardware resources are limited. Consequently, production HLS tools employ list-based scheduling heuristics that prioritize operations according to dependency constraints and resource availability [5] [7]. The resulting schedule directly determines the latency and throughput achievable by the synthesized accelerator.

C. Resource Allocation and Binding

After scheduling, the HLS compiler determines the hardware resources required to implement the scheduled operations. Resource allocation selects the number and type of functional units, while binding assigns scheduled operations to those resources. For CNN accelerators, multiply-accumulate operations are typically mapped onto dedicated DSP slices to maximize computational efficiency, whereas feature maps, weights, and intermediate activations are stored in on-chip

BRAM whenever possible. Operations scheduled at different clock cycles may share the same hardware resource, reducing area consumption, while highly parallel schedules require additional functional units. Consequently, resource allocation and binding establish the fundamental trade-off between hardware utilization and computational performance [5]. [5].

D. Architectural Optimizations through HLS Directives

Unlike conventional software compilers, HLS enables architectural optimization through synthesis directives while preserving the functional C/C++ description. These directives allow designers to control scheduling, parallelism, and memory architecture without modifying the algorithm. For CNN workloads, loop-level optimizations are particularly effective due to their MAC-intensive structure. Loop pipelining overlaps consecutive iterations and improves throughput through reduced initiation intervals (II), while loop unrolling increases spatial parallelism by executing multiple iterations concurrently at the cost of higher resource utilization. Memory optimization techniques, such as array partitioning and dataflow decomposition, further enhance performance by increasing memory bandwidth and enabling concurrent execution. Together, these optimizations enable efficient FPGA implementations of CNN accelerators.

E. Design Space Exploration

HLS enables rapid exploration of architectural trade-offs by modifying synthesis directives rather than manually rewriting RTL. Design parameters including pipeline initiation interval, loop unrolling factor, and memory architecture directly influence accelerator performance. Reducing the initiation interval increases throughput but generally requires additional hardware resources. Similarly, increasing the unrolling factor improves computational parallelism at the expense of greater DSP, LUT, and BRAM utilization. Memory optimizations improve bandwidth and reduce latency but may increase resource consumption.

The objective of HLS-based design space exploration is therefore to identify a balanced architecture that satisfies application-specific latency, throughput, timing, and resource constraints. This capability represents one of the principal advantages of HLS for Edge AI accelerator development, where multiple architectural alternatives can be evaluated within hours rather than weeks [5].

F. RTL Generation from Scheduled and Bound DFG

Following scheduling, resource allocation, binding, and architectural optimization, the HLS compiler automatically generates synthesizable RTL in VHDL or Verilog. The generated hardware consists of datapath functional units, multiplexers, pipeline registers, memory interfaces, and a finite-state machine controller that coordinates execution according to the computed schedule. The generated RTL is subsequently processed by the FPGA implementation flow, including logic synthesis, technology mapping, placement, routing, timing

analysis, and bitstream generation. The resulting FPGA accelerator is structurally equivalent to the original behavioral C/C++ specification while incorporating the hardware optimizations introduced during HLS compilation [2].

V. IMPLEMENTATION AND EVALUATION

In this initial step, a minimal 2D convolution kernel is implemented in synthesizable C/C++ as a reference design for High-Level Synthesis (HLS). The function operates on a single-channel input feature map and applies a 3×3 kernel to produce an output feature map using nested loops over spatial dimensions and kernel indices. Each output element is computed through a multiply-accumulate (MAC) operation, where partial sums are accumulated in a local register before being written back to memory. The implementation deliberately avoids any architectural optimizations at this stage, providing a clean baseline that exposes the inherent data dependencies and memory access patterns of convolution workloads.

A. Baseline Implementation Results

The baseline design implements a fully sequential 2D convolution kernel without any High-Level Synthesis (HLS) optimizations such as pipelining, loop unrolling, or memory partitioning. It serves as the reference point for evaluating all subsequent architectural improvements.

Metric	Value
Target clock period	10 ns (100 MHz)
Achieved clock period	7.11 ns
Latency	2,217,601 cycles (22.176 ms)
DSP usage	1
LUT usage	760
FF usage	260
BRAM usage	0

TABLE I
BASELINE PERFORMANCE AND RESOURCE UTILIZATION

The synthesized baseline confirms correct functional behavior, achieving timing closure at 7.11 ns and satisfying the 10 ns clock constraint. However, performance is limited by fully sequential execution, resulting in a latency of approximately 2.22 million cycles (22.176 ms). The single DSP usage indicates that all MAC operations are time-multiplexed onto one arithmetic unit, with no spatial parallelism exploited. Therefore, this baseline serves as the reference implementation for evaluating the impact of HLS optimizations, including loop pipelining, loop unrolling, and memory partitioning.

B. Pipeline Optimization Attempt

After establishing the sequential baseline, a loop pipelining optimization was applied to the inner computation of the 2D convolution kernel using Vitis HLS. The objective was to increase instruction-level parallelism by overlapping loop iterations and reducing effective computation latency. The same convolution configuration (16 input/output channels,

32×32 feature map, and 3×3 kernel) was used to ensure a fair comparison with the baseline implementation.

Metric	Value
Target clock period	10 ns (100 MHz)
Achieved clock period	7.29 ns
Latency	1,036,850 cycles (10.368 ms)
Initiation interval (II)	72 (achieved) / 1 (target)
DSP usage	144
LUT usage	7705
FF usage	3123
BRAM usage	0

TABLE II
PIPELINED PERFORMANCE AND RESOURCE UTILIZATION

The pipelining directive significantly reduced the overall latency from approximately 22.18 ms in the baseline design to 10.37 ms, achieving an improvement of roughly 2×. However, the initiation interval analysis reveals that the design does not achieve an ideal fully pipelined behavior. The achieved II of 72 indicates the presence of structural bottlenecks that prevent continuous throughput at one result per cycle. The utilization report shows a substantial increase in hardware resources, particularly in DSP usage, which increased from a single multiplier in the baseline design to 144 DSP blocks in the pipelined version. This reflects increased parallel execution of arithmetic operations introduced by the synthesis tool. However, the high II value suggests that memory access patterns and loop-carried dependencies still limit effective parallelism. Overall, while loop pipelining improves latency, the design remains constrained by memory bandwidth and data dependency limitations. This indicates that pipelining alone is insufficient for achieving an efficient hardware accelerator, and additional optimizations such as array partitioning or loop restructuring are required in subsequent design iterations.

C. Loop Unrolling for MAC-Level Parallelism

In the final optimization step, loop unrolling was applied to the input-channel dimension to introduce explicit hardware-level parallelism in the convolution operation. This transformation enables multiple multiply-accumulate (MAC) operations to be executed concurrently within a single cycle, reducing the dependency chain associated with sequential accumulation.

The results show a significant reduction in latency compared to both the baseline and pipelined designs, achieving approximately 6× improvement over the previous stage. This performance gain is directly attributed to increased parallel execution of MAC operations across input channels. Resource utilization increases moderately, particularly in DSP usage, which rises to 36 units. However, this remains within acceptable limits for Artix-7 devices, indicating a balanced trade-off between performance and hardware cost. The achieved design represents a practical edge-AI convolution accelerator with efficient parallel computation.

Metric	Value
Target clock period	10 ns (100 MHz)
Achieved clock period	6.90 ns
Latency	360,001 cycles (3.6 ms)
DSP usage	36
LUT usage	6,466
FF usage	5,691
BRAM usage	0

TABLE III
LOOP UNROLLING OPTIMIZATION RESULTS

VI. CONCLUSION

This paper presented High-Level Synthesis (HLS) as an effective methodology for implementing Edge AI accelerators on FPGA platforms. The motivation arises from the mismatch between the requirements of modern deep learning inference workloads and the strict edge limits on latency, power, memory, and deterministic. Among hardware platforms, FPGAs were identified as a suitable hardware substrate due to their reconfigurability and datapaths parallelization with high efficiency. HLS further simplifies FPGA design by translating high-level C/C++ descriptions into synthesizable RTL while exposing key optimization dimensions: scheduling, spatial parallelism, and memory architecture.

The results demonstrate that directive-driven transformations in High-Level Synthesis directly influence hardware behavior in terms of throughput, latency, and resource utilization. The synthesized architecture achieves deterministic execution with predictable timing behavior, making it suitable for real-time Edge AI inference workloads. Overall, the study indicates that HLS-based FPGA design provides an effective trade-off between performance and design complexity, particularly for compute-intensive deep learning kernels.

REFERENCES

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] Advanced Micro Devices Inc. (AMD), *Vitis High-Level Synthesis User Guide*, 2023.
- [3] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.
- [4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [5] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, and K. Bertels, "A survey and evaluation of fpga high-level synthesis tools," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, pp. 1591–1604, Oct. 2016.
- [6] S. I. Venieris and C.-S. Bouganis, "fpgaconvnet: Mapping regular and irregular convolutional neural networks on fpgas," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, pp. 326–342, Feb. 2019.
- [7] P. Coussy and A. Morawiec, *High-Level Synthesis: From Algorithm to Digital Circuit*. Dordrecht, The Netherlands: Springer, 2008.